TrafficEye: An Integrated Crowdsourced and AI-Assisted System for Traffic Violation Reporting and Enforcement

Meenakshi Garg, Shivam Chauhan, Mohit Jain, Aditya Jaiswar

Vivekanand Education Society's Institute of Technology, Mumbai 400074, India
Correspondence: meenakshi.garg@ves.ac.in; 2025.shivam.chauhan@ves.ac.in; 2025.mohit.jain@ves.ac.in; 2025.aditya.jaiswar@ves.ac.in

Abstract. Citizen-submitted photographs and videos can document traffic violations beyond the reach of fixed cameras, but each report still requires interpretation and verification. TrafficEye is a prototype that connects mobile evidence capture to an officer review interface. It records location from device GPS or available image metadata, stores evidence and report data in a managed cloud backend, and uses a pretrained vision-language model to propose a vehicle description, violation category, licence-plate string, and severity label. The reporter can check the information before submission; an officer reviews the original media and may confirm, correct, or reject the resulting report. This paper describes the integrated workflow and its implementation boundaries. It does not claim that the generated fields are accurate or that they reduce review time: application-specific testing has not yet been completed. A proposed evaluation will compare model outputs with independently labelled traffic images and clips, including difficult capture conditions. TrafficEye's contribution is a citizen-to-officer reporting design with AI-assisted triage and retained human authority, rather than a new recognition algorithm.

Keywords: Traffic violation reporting , Mobile crowdsourcing , Vision-language models , Automatic number plate recognition , Geolocation , Human-in-the-loop AI , Intelligent transportation systems

1 Introduction

Fixed cameras and police patrols cannot observe every road event. Reports from road users offer another source of evidence, as crowdsourced traffic and road-safety systems have demonstrated [1–4]. Indian police applications already accept citizen-submitted media for verification [5, 6]. Yet a photograph and location alone do not tell a reviewer which vehicle is involved, what violation is alleged, or whether a plate is legible. Manual entry places much of that work on the reporter; manual interpretation shifts it to the reviewing officer.

TrafficEye connects capture, metadata handling, visual interpretation, and officer verification in one prototype. A hosted vision-language model (VLM) generates

candidate fields, not enforcement decisions. The system-level contribution is the workflow and its explicit separation of machine suggestions from officer judgement. Recognition quality, review-time savings, and operational suitability remain open questions.

2 Related Work

RoadCop separates incoming citizen reports from expert evaluation and includes spam screening [1]. CrowdOut supports mobile offence reporting and map-based presentation [2]. Mobile Roadwatch attaches time and location to smartphone video before a user selects an event to report [7]. These systems show how media and context can enter a review process; none of those descriptions establishes that every visual field required by TrafficEye is extracted automatically. Work on reported danger spots [3] and smartphone sensing of hazardous road sections [4] addresses wider road-safety observation rather than the interpretation of an individual alleged offence.

Delhi's Traffic Prahari requests the date, time, location, registration number, and violation type alongside a photograph or video, subject to police verification [5]. The Mumbai Traffic Police app also provides a citizen-reporting route [6]. TrafficEye follows the same principle of authority-led review while proposing machine-generated fields for the initial report. The cited product descriptions do not establish that those services lack similar features; the distinction here is the documented integration in this prototype.

Task-specific methods approach interpretation differently. Srivastava et al. combine YOLOv5 with EasyOCR for Indian number plates [8], while Deshpande et al. describe a staged helmet-violation and vehicle-identification pipeline [9]. Surveys of automatic number plate recognition (ANPR) describe the effects of detection, OCR, image quality, viewpoint, and environmental conditions on evaluation [10, 11]. These studies provide task definitions and measurement criteria; their reported performance cannot be transferred to TrafficEye. A single prompted VLM avoids training a project-specific detector but gives the application less control over each output field. Surveys of vision-language and multimodal models [12, 13] motivate this reusable inference approach, and the hosted API documents image input [14], but none verifies its traffic-enforcement accuracy.

Public acceptance and governance matter as much as interface design. Privacy, perceived penalty fairness, and trust in police influence acceptance of crowdsourced enforcement [15]. A review of AI use in law enforcement stresses human involvement and accountability [16]. Accordingly, TrafficEye presents original evidence with the suggested fields and leaves confirmation to an officer.

3 Problem Statement

The engineering problem is to turn heterogeneous citizen media into a reviewable traffic-violation report without treating automated interpretation as verified evidence. Three linked difficulties define it: (i) reporters must provide usable media, time, location, vehicle details, and an alleged violation, although manual entry is slow and error-

prone; (ii) officers must still inspect the media, read any visible plate, and resolve inconsistent or incomplete fields; and (iii) dedicated models for separate plate and violation tasks require task-specific development, while general-purpose visual models have not been validated for this application [8–14]. A reporting system therefore needs to combine evidence collection, cautious location handling, candidate visual fields, and an auditable officer decision. TrafficEye addresses that integration gap; it does not establish that the proposed automation is reliable until domain-specific evaluation is performed.

4 Methodology and Workflow

The reporter follows a capture–check–submit sequence. The mobile client accepts a live photograph or video, or existing media. It obtains device GPS coordinates for live capture and reads available EXIF fields from gallery images. The reporter checks the location and proposed report details before submission. The backend stores the original evidence and metadata; hosted inference returns candidate fields for the report. In the officer interface, the original media remains visible beside those fields. An officer can confirm, edit, or reject the submission, and the shared report status is updated. Figure 1 summarizes the two human interfaces and the storage and inference paths.

Citizen capture + GPS/EXIF →	Cloud media + report record →	Hosted VLM candidate fields →	Officer verifies

Fig. 1. Conceptual TrafficEye architecture: citizen capture and location handling, managed evidence and report storage, hosted VLM inference, and officer verification. (Diagram reproduced from the described workflow; not an evaluation result.)

The planned evaluation will use approximately 50–100 traffic images and short clips covering supported categories such as no helmet, red-light running, and wrong parking. The collection is intended only for application testing, not model training. It will include favourable images and cases with low light, oblique viewpoints, or partially obscured plates. Three project members will independently label vehicle presence, violation category, visible plate text, and a subjective severity category, then resolve disagreements by consensus. With model and prompt settings held constant, outputs will be compared with these reference labels. Proposed measures are violation-classification accuracy, vehicle-detection recall, exact plate-string match, character-level plate accuracy, and severity agreement. This is a protocol, not a completed experiment; sample size, category balance, and agreement procedures should be finalized before testing.

5 System Architecture and Implementation

The mobile client uses a cross-platform framework for Android and iOS and accesses camera, location, and local media. For live capture, it records device coordinates. For gallery images, it reads any available EXIF geotags and capture time and compares the stored position with the device's current position as a plausibility check. EXIF may be

missing, stripped, or edited, so neither it nor the comparison proves where a photograph was taken. The reporter can inspect the displayed location before submission.

The managed backend stores media objects and relational report records. A report connects evidence and location information to candidate analysis fields, severity, and review status. Subscriptions update the officer queue when new reports arrive or statuses change. Status, location, and severity can be used for filtering and triage; a severity suggestion is not a finding of guilt or an automatic enforcement priority. Backend throughput and concurrent-use behaviour have not been load-tested. Upload and hosted inference also require network access.

Before inference, the system compresses and normalizes submitted evidence. Images or selected video frames are sent to the hosted VLM with a task-specific request for structured output [14]. The response is parsed into a vehicle description, candidate violation, plate string, and one of the configured severity categories (Low, Medium, High, Critical). Plate extraction may use the same response or a follow-up call. Figure 2 shows this inference-only path. TrafficEye does not train, fine-tune, or modify the pretrained model. Model output is stored with the report for officer correction, while the original media remains the evidentiary item.

Submitted media →	Compress / normalize →	VLM inference →	Parse fields →	Officer review

Fig. 2. Conceptual inference path from normalized media to structured candidate fields and officer review; no model training is involved.

6 Prototype Outcome and Discussion

The present result is an integrated prototype: a citizen can submit media with location context, the backend can prepare a structured report using hosted visual inference, and an officer can inspect and act on that report. Figures 3 and 4 show the pending-review queue and an example of the AI-assisted review interface. The displayed model output is an interface example, not a measured accuracy or calibrated confidence result.

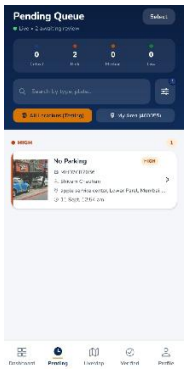


Fig. 3. Prototype officer queue showing pending reports and review-oriented filtering.

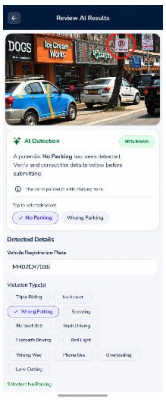


Fig. 4. Prototype AI-assisted review screen with candidate violation and plate fields available for officer correction.

The prototype establishes feasibility of connecting these components, but not their effectiveness in traffic enforcement. No domain-specific accuracy, vehicle recall, plate-recognition score, severity agreement, reporting-completion rate, or officer time saving has been measured. Low light, rain, motion blur, obstructed plates, and ambiguous events may produce incorrect or incomplete suggestions [10–13]. The current location check is not tamper-proof. No automatic bystander anonymization is implemented, and poor connectivity can interrupt the workflow. These limitations make officer review and a prospective privacy assessment necessary [15, 16]. Future tests should also record correction frequency and review time, rather than relying on field-level recognition alone.

7 Conclusion

TrafficEye brings citizen media capture, location context, cloud report handling, and pretrained visual inference into one officer-reviewed reporting workflow. Its contribution is the integration of these functions and the clear boundary between candidate machine output and an officer's decision. The prototype demonstrates the workflow; it does not yet demonstrate reliable recognition or a reduction in review effort. Those claims depend on the planned evaluation.

8 Future Work

The first priority is to complete the labelled, domain-specific evaluation and measure both field accuracy and officer correction effort under varied capture conditions. Subsequent work can add precinct or radius-based queues, a city-wide heatmap, stronger location checks, automatic masking of bystanders and unrelated vehicles, and backend load tests. A location-confidence score would need independent validation before evidentiary use. Registration lookup, evidence-linked digital challans, and owner

notification would require authorized integration with the relevant public systems; these functions are proposals, not current features.

Acknowledgments.

The authors thank Vivekanand Education Society's Institute of Technology, Mumbai, for academic support and resources.

Disclosure of Interests.

The authors declare no competing interests relevant to this article.

9 References

1. Jameela, M., Afzal, H., Khurshid, K., Malik, A.W.: Crowdsourced system to report traffic violations—RoadCop: bi-modular system. In: VEHITS 2018, pp. 315–322 (2018). https://doi.org/10.5220/0006680303150322
2. Aubry, E., Silverston, T., Lahmadi, A., Festor, O.: CrowdOut: a mobile crowdsourcing service for road safety in digital cities. In: PerCom Workshops, pp. 86–91. IEEE (2014). https://doi.org/10.1109/PerComW.2014.6815170
3. Olma, J., Bode, T., Ehlers, J., Sutter, C.: Road users' reports on danger spots: the crowd as an underestimated expert? Safety 8(4), 70 (2022). https://doi.org/10.3390/safety8040070
4. Dunlop, M.D., Roper, M., Elliott, M., McCartan, R., McGregor, B.: Using smartphones in cities to crowdsource dangerous road sections and give effective in-car warnings. In: SEACHI 2016, pp. 14–18. ACM (2016). https://doi.org/10.1145/2898365.2899796
5. Delhi Traffic Police: Traffic Prahari citizen reporting. Delhi Traffic Police Newsletter (Apr 2025). https://www.traffic.delhipolice.gov.in/sites/default/files/uploads/2025/NEWSLETTER-APRIL-2025.pdf
6. Mumbai Traffic Police: Mumbai Traffic Police App (MumTrafficapp). Apple App Store. https://apps.apple.com/in/app/mumbai-traffic-police-app/id1450958843 (accessed 4 Oct 2026)
7. Korea Advanced Institute of Science and Technology: Improving traffic safety with a crowdsourced traffic violation reporting app (Mobile Roadwatch). EurekAlert (2017). https://www.eurekalert.org/news-releases/763943
8. Srivastava, A., Narote, P., Fatangare, S., Kakade, S., Kulkarni, R.: Automatic number plate detection system for Indian vehicles using YOLOv5 and EasyOCR. In: Soft Computing for Problem Solving, pp. 305–314. Springer (2024). https://doi.org/10.1007/978-981-97-3292-0_21
9. Deshpande, U.U., et al.: Computer-vision based automatic rider helmet violation detection and vehicle identification in Indian smart city scenarios using NVIDIA TAO toolkit and YOLOv8. Front. Artif. Intell. 8, 1582257 (2025). https://doi.org/10.3389/frai.2025.1582257
10. Shashirangana, J., Padmasiri, H., Meedeniya, D., Perera, C.: Automated license plate recognition: a survey on methods and techniques. IEEE Access 9, 11203–11225 (2021). https://doi.org/10.1109/ACCESS.2020.3047929
11. Xu, L., Shang, Z., Chen, X., Zeng, T., Dai, W., Zhao, J.: Deep learning algorithms for license plate recognition: a review. Neural Networks 200, 108842 (2026). https://doi.org/10.1016/j.neunet.2026.108842
12. Zhang, J., Huang, J., Jin, S., Lu, S.: Vision-language models for vision tasks: a survey. IEEE Trans. Pattern Anal. Mach. Intell. 46(8), 5625–5644 (2024). https://doi.org/10.1109/TPAMI.2024.3369699

13. Caffagni, D., et al.: The revolution of multimodal large language models: a survey. In: Findings of ACL 2024, pp. 13590–13618 (2024). https://doi.org/10.18653/v1/2024.findings-acl.807
14. Google: Gemini API: image understanding. Google AI for Developers. https://ai.google.dev/gemini-api/docs/image-understanding (accessed 4 Oct 2026)
15. Khojastehpour, M., Sahebi, S., Samimi, A.: Public acceptance of a crowdsourcing platform for traffic enforcement. Case Stud. Transp. Policy 10(4), 2012–2024 (2022). https://doi.org/10.1016/j.cstp.2022.08.013
16. Dasgupta, R., et al.: Unlocking Australia's AI usage in law enforcement from human involvement perspective: a systematic literature review. AI Soc. 40, 6543–6570 (2025). https://doi.org/10.1007/s00146-025-02350-6